Chapter 6

Distributed Database Systems

Outline

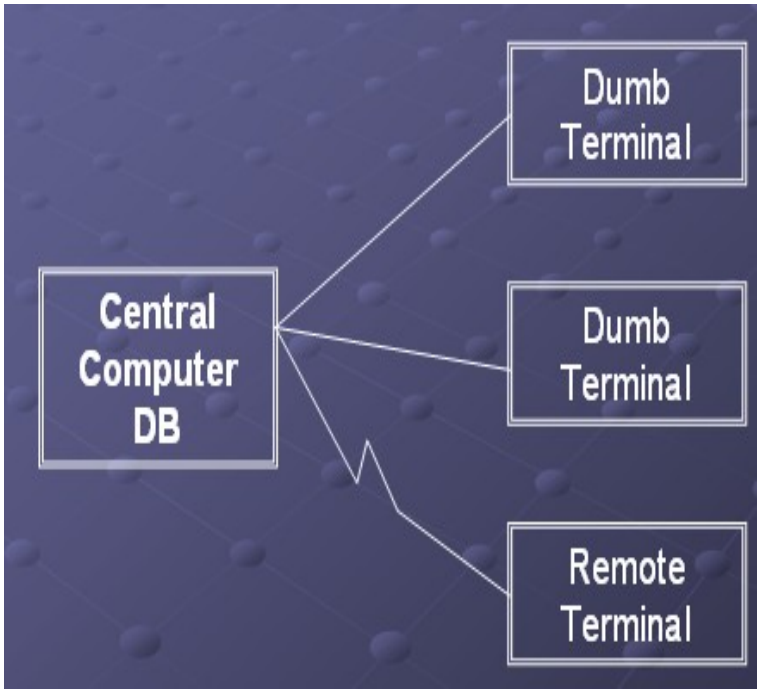
- Distributed Database Concepts
- Data Fragmentation, Replication and Allocation
- Types of Distributed Database Systems
- Query Processing
- Concurrency Control

Distributed Database Concepts

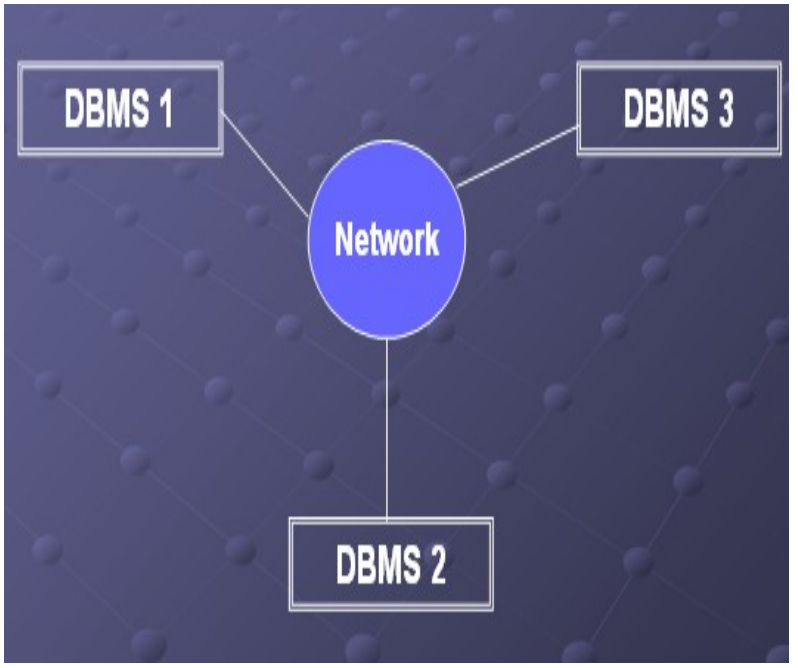
- A transaction can be executed by multiple networked computers in a unified manner.
- A **distributed database (DDB)** processes unit of execution (a transaction) in a distributed manner.
- A distributed database (DDB) can be defined as :
  - A distributed database (DDB) is a collection of multiple logically related database distributed over a computer network, and a distributed database management system as a software system that manages a distributed database while making the distribution transparent to the user.
  - *Distributed Database is not a centralized database.*

…CON'T

- Centralized Database Vs. Distributed Database



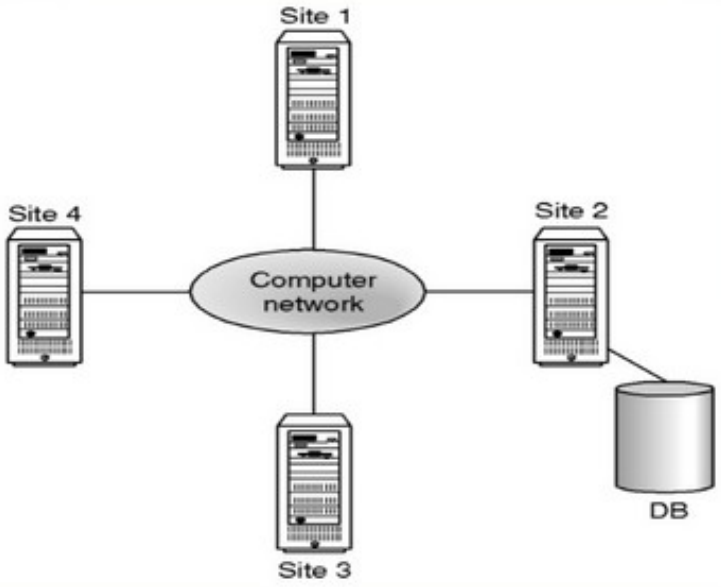
Centralized DB



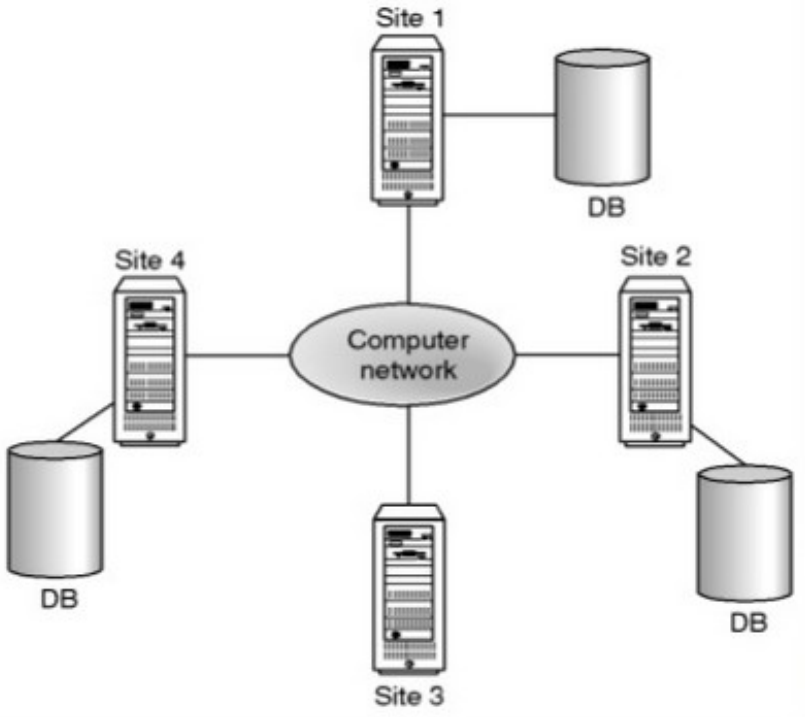
Distributed DB

…CON'T

➔ Centralized Database



➔ Distributed Database



Data allocation

- *is the process of deciding where to allocate/store particular data item.*
- *There are 3 data allocation strategies:*
  1. ***Centralized:*** *the entire DB is located at a single site. And computers access through the network. Known as distributed processing.*
  2. ***Partitioned:*** *the DB is split into several disjoint parts (called partitions, segments or fragments) and stored at several sites*
  3. ***Replicated****: copies of one or more partitions are stored at several sites:*

     3.1 ***Selective-*** combines *fragmentation (locality of reference for those which are less updated) replication and centralization as appropriate for the data.*

     3.2 ***Complete: -*** *database copy is made available in each site. Snap shot is one method used here.*

…CON'T

- In a distributed database system, the database is logically stored as single database but physically fragmented on several computers.
- The computers in a distributed system communicate with each other through various communication media, such as high speed buses or telephone line.
- A distributed database system has the following components:
  - Local DBMS
  - Distributed DDBMS
  - Global System Catalog(GSC)
  - Data communication (DC)

…CON'T

- Distributed database system consists of a collection of sites, each of which maintains a local database system (Local DBMS) but each local DBMS also participates in at least one global transaction where different databases are integrated together.
  - ***Local Transaction***: transactions that access data only in that single site
  - ***Global Transaction***: transactions that access data in several sites.
- ***Parallel DBMS:*** a DBMS running across multiple processors and disks that is designed to execute operations in parallel, when ever possible, in order to improve performance.

…CON'T

- Three architectures for parallel DBMS:
  - Shared Memory- for fast data access for a limited number of processors.
  - Shared Disk- for application inherently centralized
  - Shared nothing.- massively parallel
- What makes DDBMS different is that
  - The various sites are aware of each other
  - Each site provides a facility for executing both local and global transactions.
- The different sites can be connected physically in different topologies like:
  - ***Fully /networked,***
  - ***Partially Connected,***
  - ***Tree Network,***
  - ***Star Network and***
  - ***Ring Network***

…CON'T

- The differences between these sites is based on:
  - ***Installation Cost***: cost of linking sites physically.
  - ***Communication Cost***: cost to send and receive messages and data
  - ***Reliability***: resistance to failure
  - ***Availability***: degree to which data can be accessed despite the failure.

…CON'T

- The distribution of the database sites could be:
  - **Large Geographical Area: *Long-Haul Network***
    - *relatively slow*
    - *less reliable*
    - *uses telephone line, microwave, satellite*
  - **Small Geographical Area: Local Area Network**
    - *higher speed*
    - *lower rate of error*
    - *use twisted pair, base band coaxial, broadband coaxial, fiber optics*

…CON'T

- Distributed database system consists of loosely joined sites that share no physical component and database systems that run on each site are independent of each other.
- Those which share physical components are known as Parallel DBMS.
- Transactions may access data at one or more sites
- Organization may implement their database system on a number of separate computer system rather than a single, centralized mainframe.
- Computer Systems may be located at each local branch office.

Functions of a DDBMS

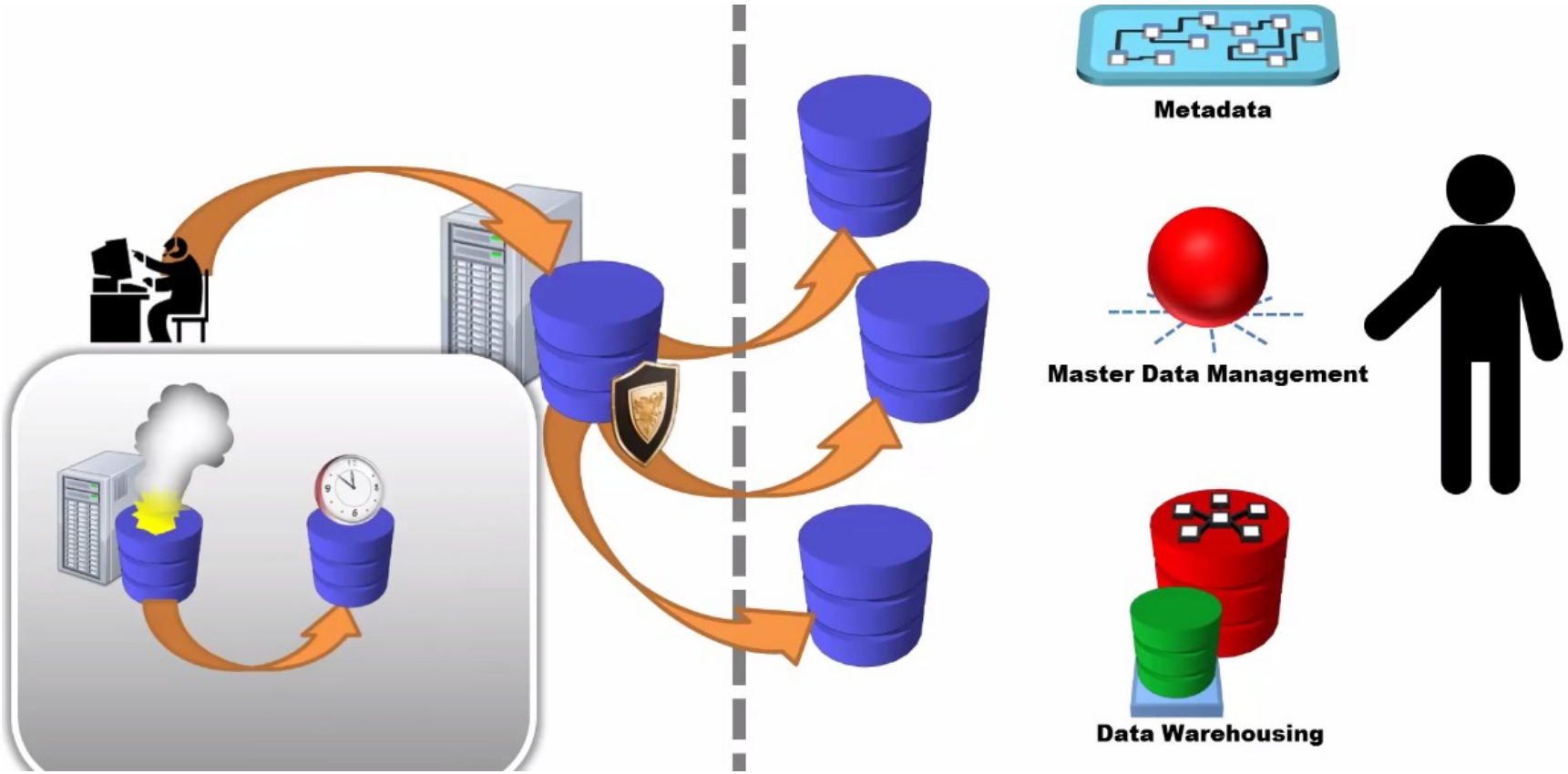
- DDBMS have the following functionality.
  - Extended Communication Services - to provide access to remote sites.
  - Extended Data Dictionary- to store data distribution details a need for global system catalog.
  - Distributed Query Processing - optimization of query remote data access.
  - Extended security- access control to a distributed data
  - Extended Concurrency Control –maintain consistency of replicated data.
  - Extended Recovery Services- failures of individual sites and the communication line.

Issues in DDBMS

- **How is data stored in DDBMS?**
  - There are several ways of storing a single relation in distributed database systems.
  - **Replication:**
    - System maintains multiple copies of similar data (identical data)
    - Stored in different sites, for faster retrieval and fault tolerance.
    - Duplicate copies of the tables can be kept on each system (replicated).
    - Updates to the tables can become involved (of course the copies of the tables can be read-only).
    - Advantage: Availability, Increased parallelism (if only reading)-
- Disadvantage: increased overhead of update.

…CON'T

- Replication



…CON'T

- **Fragmentation:**
  - Relation is partitioned into several fragments stored in distinct sites
    - The partitioning could be ***vertical***, ***horizontal*** or ***both***.
  - **Horizontal Fragmentation**
    - Systems can share the responsibility of storing information from a single table with individual systems storing groups of rows
    - Performed by the ***Selection Operation***
    - The whole content of the relation is reconstructed using the ***UNION*** operation.

$$\sigma_{\text{BUDGET}<200000}(\text{PROJ})$$

$$\sigma_{\text{BUDGET}\geq 200000}(\text{PROJ})$$

…CON'T

- **Example (contd.):** Horizontal fragmentation of PROJ relation
  - – PROJ1: projects with budgets less than 200, 000
  - – PROJ2: projects with budgets greater than or equal to 200, 000

PROJ

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston

$PROJ_1$

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York

$PROJ_2$

PNO	PNAME	BUDGET	LOC
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston

…CON'T

- **Vertical Fragmentation**
  - Systems can share the responsibility of storing particular attributes of a table.
  - ***Needs attribute with tuple number (the primary key value be repeated.)***
  - Performed by the ***Projection Operation***
  - The whole content of the relation is reconstructed using the ***Natural JOIN*** operation using the attribute with ***Tuple number (primary key values).***
- **Example:**
  - PROJ1 = πPNO,BUDGET (PROJ)
  - PROJ2 = π PNO,PNAME,LOC(PROJ)

…CON'T

- **Example:** Vertical fragmentation of PROJ relation
  - PROJ1: information about project budgets
  - PROJ2: information about project names and locations

PROJ

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston

$PROJ_1$

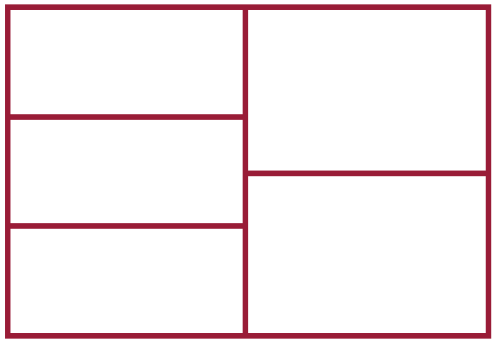
PNO	BUDGET
P1	150000
P2	135000
P3	250000
P4	310000
P5	500000

$PROJ_2$

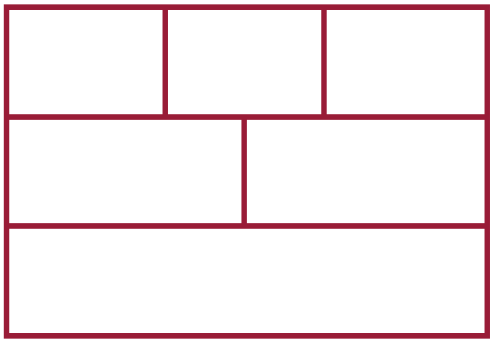
PNO	PNAME	LOC
P1	Instrumentation	Montreal
P2	Database Develop.	New York
P3	CAD/CAM	New York
P4	Maintenance	Paris
P5	CAD/CAM	Boston

…CON'T

- ***Both (hybrid fragmentation)***
  - *A system can share the responsibility of storing **particular attributes** of a **subset of records** in a given **relation**.*
  - *Performed by projection then selection or selection then projection relational algebra operators.*
  - *Reconstruction is made by combined effect of **Union** and **natural join** operators.*



(a)



(b)

…CON'T

- ***Fragmentation is correct if it fulfils the following***
  - ***Complete****: - a data item must appear in at least one fragment of a given relation R (R1, R2…Rn).*
  - ***Reconstruction****:- it must be possible to reconstruct a relation from the fragments*
  - ***Disjointness****: - a data item should only be found in a single fragment except for vertical fragmentation (the primary key is repeated for reconstruction).*

Data transparency:

- The degree to which system user may remain unaware of the details of how and where the data items are stored in a distributed system.
  - **Distribution transparency** Even though there are many systems they appear as one- seen as a single, logical entity.
  - **Replication transparency** Copies of data floating around everywhere also seem like just one copy to the developers and users
  - **Fragmentation transparency** A table that is actually stored in parts everywhere across sites may seem like just a single table in a single
  - **Location Transparency-** the user doesn't need to know where a data item is physically located.

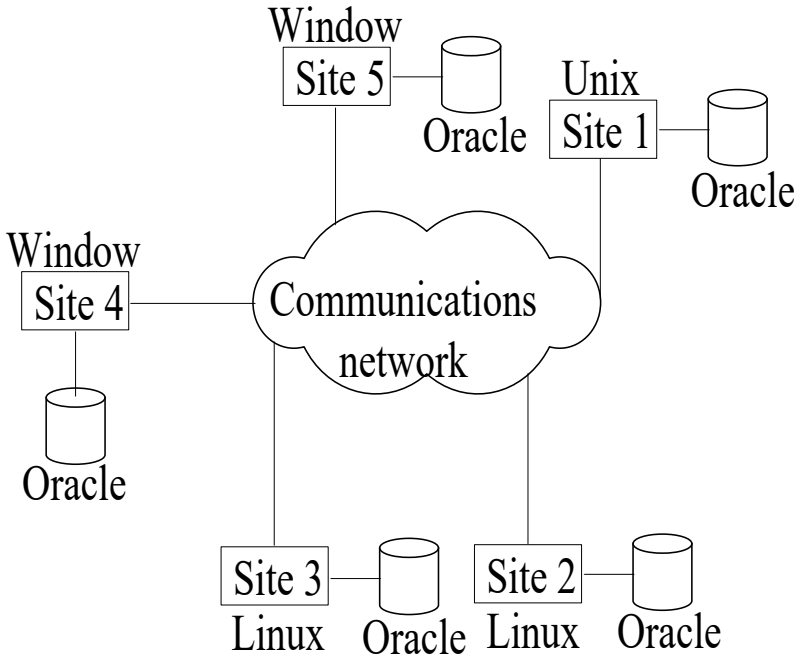
…CON'T

- ***How does it work?***

- Distributed computing can be difficult to implement, particularly for replicated data that can be updated from many systems.
- In order to operate a distributed database system has to take care of:
  - **Distributed Query Processing**
  - **Distributed Transaction Management**
  - **Replication Data Management** If you are going to have copies of data on many machines how often does the data get updated if it is changed in another system? Who is in charge of propagating the update to the data?
  - **Distributed Database Recovery** If one machine goes down how does that affect the others.
  - **Security:** Just like any computer network, a distributed system needs to have a common way to validate users entering from any computer in the network of servers.
  - **Common Data-Dictionary** Your schema now has to be distinguished and work in connection to schemas created on many systems.

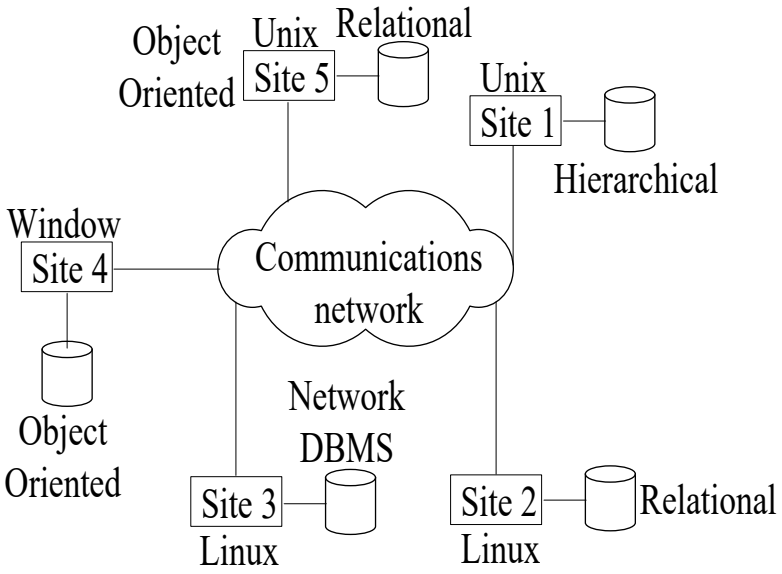
Homogeneous and Heterogeneous Distributed Databases

- **In a homogeneous distributed database**
  - Are aware of each other and agree to cooperate in processing user requests.
  - Each site surrenders part of its autonomy in terms of right to change schemas or software
  - Appears to the user as a single system
  - All sites of the database system have identical setup, i.e., same database system software.
  - The underlying operating system may be different.
    - For example, all sites run Oracle or DB2, or Sybase or some other database system.
  - The underlying operating systems can be a mixture of Linux, Window, Unix, etc.



…CON'T

- **In a heterogeneous distributed database**
  - Different sites may use different schemas and software (DBMS)
    - Difference in schema is a major problem for query processing
    - Difference in software is a major problem for transaction processing
  - Sites may not be aware of each other and may provide only limited facilities for cooperation in transaction processing.
  - May need ***gateways*** to interface one another.



Why DDBMS/Advantages

- **Many existing systems**
  - Possibly there are many different existing system, with possible different kinds of systems (Oracle, Informix, others) that need to be used together.
- **Data sharing and distributed control:**
  - User at one site may be able access data that is available at another site.
  - Each site can retain some degree of control over local data
  - We will have local as well as global database administrator
- **Reliability and availability of data**
  - If one site fails the rest can continue operation as long as transaction does not demand data from the failed system and the data is not replicated in other sites
- **Speedup of query processing**
  - If a query involves data from several sites, it may be possible to split the query into sub-queries that can be executed at several sites which is parallel processing
  - Query can be sent to the least heavily loaded sites
  - **Expansion** (Scalability)
  - In a distributed environment you can easily expand by adding more machines to the network.

Disadvantages of DDBMS

1. **Software Development Cost**
   - Is difficult to install, thus is costly
2. **Greater Potential for Bugs**
   - Parallel processing may endanger correctness of algorithms
3. **Increased Processing Overhead**
   - Exchange of message between sites – high communication latency
   - Due to communication jargons
4. **Communication problems**
5. **Increased Complexity and Data Inconsistency Problems**
   - Since clients can read and modify closely related data stored in different database instances concurrently.
6. **Security Problems:** network and replicated data security.

Query Processing and Transaction Management in DDBMS

Query Processing

- There are different strategies to process a specific query, which in turn increase the performance of the system by minimizing processing time and cost.
- In addition to the cost estimates we have for a centralized database (disk access, relation size, etc), we have to consider the following in distributed query processing:
  - Cost of data transmission over the huge network
  - Gain of parallel processing of a single query

…CON'T

- Let the distributed database has three sites (S1, S2, and S3). And two relations, EMPLOYEE and DEPARTMENT are located at S1 and S2 respectively without any fragmentation. And a query is initiated from S3 to retrieve employees [First Name (15 byte long), Last name (15 byte long) and Department name (10 byte long) total of 40 bytes with the department they are working in.
- Let:
- For EMPLOYEE we have the following information
  1. 10,000 records
  2. each record is 100 bytes long
- For DEPARTMENT we have the following information
  1. 100 records
  2. each record is 35 bytes long

…CON'T

- There are three ways of executing this query:
  1. Transfer DEPARTMENT and EMPLOYEE to S3 and perform the join there: needs transfer of 10,000*100+100*35=1,003,500 byte.
  2. Transfer the DEPARTMENT to S1, perform the join there which will have 40*10,000 = 400,000 bytes and transfer the result to S3. we need 1,000,000+400,000=1,400,000 byte to be transferred
  3. Transfer the EMPLOYEE to S2, perform the join there which will have 40*10,000 = 400,000 bytes and transfer the result to S3. We need 3,500+400,000=403,500 byte to be transferred.
- Then one can select the strategy that will reduce the data transfer cost for this specific query.
- Other steps of optimization may also be included to make the processing more efficient by *reducing the size of the relations using projection*.

Transaction Management

- Transaction is a logical unit of work constituted by one or more operations executed by a single user.
- A transaction begins with the user's first executable query statement and ends when it is committed or rolled back.
- A **Distributed Transaction** is a transaction that includes one or more statements that, individually or as a group, update data on two or more distinct nodes of a distributed database.
- **Representation of Query in Distributed Database**

SQL Statement	Object	Database	Domain
SELECT * FROM dept@sales.midroc.telecom.et;	dept	sales	midroc.telecom.et ;

…CON'T

- There are two types of transaction in DDBMS to access data from other sites:

1. **Remote Transaction:** contains only statements that access a single remote node.
   - **Remote Query** statement is a query that selects information from one or more remote tables, all of which reside at the same remote node or site.
   - For example, the following query accesses data from the *dept* table in the *Addis* schema (the site) of the remote sales database:
     - SELECT * FROM Addis.dept@sales.midroc.telecom.et;

- A remote **update** statement is an update that modifies data in one or more tables, all of which are collocated at the same remote node.
- For example, the following query updates the branch table in the Addis schema of the remote sales database:

```
UPDATE Addis.dept@ sales.midroc.telecom.et;
SET loc = 'Arada'
WHERE BranchNo = 5;
```

…CON'T

2. **Distributed Transaction:** contains statements that access more than one node.

- A distributed query statement retrieves information from two or more nodes.
- If all statements of a transaction reference only a single remote node, the **transaction is remote, not distributed**.
- A database must guarantee that all statements in a transaction, distributed or non-distributed, either commit or roll back as a unit.
- **For example**, the following query accesses data from the local database as well as the remote sales database:

```
SELECT ename, dname
FROM Awassa.emp AW, Addis.dept@ sales.midroc.telecom.et AD
WHERE AW.deptno = AD.deptno;
```

{Employee data is stored in Awassa and Sales data is stored in Addis, there is an employee responsible for each sale }

…CON'T

- **Remote query**

```
select
client_nm
from
 clients@accounts.motorola.com;
```

- **Distributed query**

```
select
 project_name, student_nm
from
 intership@accounts.motorola.com i,
student s
where
s.stu_id = i.stu_id
```

- **Remote Update**

```
update intership@accounts.motorola.com
 set
  stu_id = '242'
  where stu_id = '200'
```

- **Distributed Update**

```
update
intership@accounts.motorola.com set
  stu_id = '242'
Where stu_id = '200'
  update student
Set  stu_id = '242` where  stu_id = '200`
commit
```

Concurrency Control

- There are various techniques used for concurrency control in centralized database systems.
- The techniques in *distributed database system* are similar with the *centralized approach* with additional implementation requirements or modifications.
- The main difference or the change that should be incorporated is the way the lock manager is implemented and how it functions.
- There are different schemes for concurrency control in DDBS

…CON'T

1) Non-Replicated Scheme

- No data is replicated in the system
- All sites will maintain a local lock manager (local lock and unlock)
- If site Si needs a lock on data in site Sj it send message to lock manager of site Sj and the locking will be handled by site Sj
- All the locking and unlocking principles are handled by the local lock manager in which the data object resides.
- Is simple to implement
- Need three message transfers
  - To request a lock
  - To notify grant of lock
  - To request unlock

…CON'T

2) Single Coordinate Approach

- The system choose one single lock manager that resides in one of the sites (S_i)
- All locks and unlocks requests are made at site Si where the lock manager resides(S_i)
- Is simple to implement
- Needs two message transfers
  - To request a lock
  - To request unlock
- Simple deadlock handling
- Could be a bottleneck since all processes are handled at one site
- Is vulnerable/at risk if the site with the lock manager fails

…CON'T

- There are three varieties of the 2PL (Two phase locking) protocol in the DDBMS environment.
- Implementing the basic 2PL in distributed systems assumes that data is distributed across multiple machines.
- *Centralized 2PL:*
  - A single site responsible for granting and releasing locks
  - Each site's transaction manager communicates with this centralized lock manager and with its own local data manager
  - Has **only one** lock manager for the entire site.

…CON'T

- *Primary 2PL:*
  - Each replicated data item is assigned a ***primary copy*;**
    - the lock manager on that primary copy is responsible for granting and releasing locks(distributed locking) and updates are propagated as soon as possible to the **slave copies**
  - Distributes the lock manager to ***a number of sites***
- *Distributed 2PL:*
  - Assumes data is completely **replicated**
  - The schedulers (Lock managers) at each site are responsible in granting and releasing locks as well as forwarding operations to the local data manager.
  - Distributes the lock manager to **every site** in the DDBS.
  - Has ***communication overhead*** than the others.

END